

Gemini:

1. Map-Generierung: Vollprozedural vs. Modulare Bausteine

Vollprozedural generierte 3D-Gräben per Perlin-Noise oder marching cubes sind eine Falle. Sie erzeugen hässliche Geometrie, kaputtes Pathing und verheerendes FP-Leveldesign (Sichtlinien-Fehler, unspielbare Sackgassen).

- **Option A: Fully Procedural (Mesh-Gen):** Maximaler Aufwand, hohes Risiko für Desorientierung und unfaire Sichtlinien.
- **Option B: Modular Stacking (Snap-to-Grid / Tile-basiert):** Vordefinierte Block-Typen (T-Stück, 90°-Kurve, Bunker, Parapet, Unterstand) auf einem fixen Rastersystem.

Empfehlung: Option B (Modulares Tile-System). Nutze ein festes Grid (z. B. 4x4m Blöcke). Der Generator wählt aus vorgefertigten Bauzeichen.

- **Lesbarkeit:** Verwende das **3-Farben-Kabel-Prinzip** (wie *Left 4 Dead* oder *Deep Rock Galactic*). Ein fettes, dickes Telefonkabel verläuft am Boden/an der Grabenwand durchgehend von der Front bis zur Home-Line. Spieler in Panik schauen auf den Boden und folgen der Kabelfarbe.
- **Skalierung:** Skaliere für 1–4 Spieler **nicht die Breite** des Sektors (das zerstört das Graben-Gefühl und die Sichtachsen), sondern die **Länge der Front** (z. B. 2 aktive Grabenabschnitte bei Solo, 4 bei 4 Spielern) und die Dichte der Gegnerwellen.

2. Gegner-KI & Pathing: Wie viel Tiefe ist nötig?

- *Option A: Vollständiges NavMesh (A) für jeden Gegner:** Teuer im Browser bei Horden (Nacht-Modus), erfordert komplexe Re-Calculations bei zerstörbarer Deckung.
- **Option B: Hybrid – Graph-basiertes Waypoint-Netzwerk im Graben + Directional Steering im Niemandsland:**
 - **Niemandsland:** Gegner laufen direkt auf die Frontlinie zu (Raycast-Hindernisvermeidung reicht).
 - **Im Graben:** Ein vereinfachter Graph (Knotenpunkte an Grabenkreuzungen). Gegner wählen am Knoten den kürzesten Pfad zum nächsten Spieler oder Ziel.

Empfehlung: Option B. Erstelle das Nav-Netzwerk automatisch aus dem modularen Grid-System (Option B bei den Maps liefert die Graph-Knoten gratis mit!).

- **KI-Tiefe:** Du brauchst **keine** komplexe Verhaltensbaum-KI. Drei Basis-Verhalten reichen völlig:
 1. *Runner/Charger:* Ignoriert Deckung, rennt zum Ziel (Wiedergänger, Bajonett-Charger).
 2. *Suppressor:* Läuft bis Sichtlinie, stützt sich ab, feuert Salven (MG-Trupp).
 3. *Breacher:* Sucht gezielt platzierte Verteidigung/Draht und zerstört diese mit Sprengstoff (Sturmtrupp, Grabenräumer).
- *Referenz: Vermintide 2* – Gegner wirken klug, weil sie auditive Signals (Heulen, Rüstungsrascheln) senden und in Formationen auftreten, nicht weil ihre KI hochkomplex ist.

3. Der Rückzugs-Loop & In-Mission-Quests

Deine Idee mit der In-Mission-Quest beim Feldkommandeur für die aktive Fähigkeit birgt ein massives **Design-Dilemma:**

1. **Der Rückzugs-Loop:** Wenn die Frontlinie fällt, ziehst du dich zurück. Das ist exzellente Dynamik (*Hell Let Loose* offensive mode, *Helldivers 2*).
2. **Fähigkeiten-Quest beim Feldkommandeur:** Wenn Spieler mitten im Gefecht zur Home-Line zurücklaufen müssen, um eine Quest abzugeben und ihre Fähigkeit freizuschalten, verlassen sie die Front genau dann, wenn sie am nötigsten gebraucht wird.

Kritik & Empfehlung: Streiche die Hol-und-Bring-Quest beim Feldkommandeur. Sie bremst den Spielfluss. Schalte Fähigkeiten stattdessen dynamisch über den **Wellen-Fortschritt** oder erbrachte Leistungen frei (z. B. "30 Gegner besiegt" oder "Welle 2 überstanden").

Rückzug lesbar machen: Nutze klare akustische und visuelle Signale:

- Signalpfeife (Audio-Cue) ertönt.
- Ein Countdown läuft ab, bevor die Frontlinie von feindlichem Sperrfeuer/Gas eingenommen wird (verhindert Bunkern).
- Ein **Sperrfeuer-Vorhang** rückt nach hinten nach – wer zu lange wartet, stirbt im Artillerieregen.

4. Nachschub-Ökonomie

- **Option A: Zwei Währungen (In-Game vs. Meta):** Z. B. *Munition* im Einsatz für Barrikaden/Fähigkeiten und *Ressourcen* nach der Extraktion fürs Quartier.
- **Option B: Eine universelle In-Einsatz-Ressource ("Baumaterial/Nachschub"):**

Empfehlung: Option B im Einsatz, strikt getrennt vom Quartier. Verwende während des Einsatzes **nur eine In-Game-Währung: Nachschubpunkte** (erhalten durch Kill-Boni, Wellenabschluss, Versorgungskisten im Niemandsland).

- **Kein flaches Entscheidungsproblem:** Die Tiefe entsteht durch den Zielkonflikt: *Kaufe ich jetzt sofort Munition/Heilung für mich selbst ODER baue ich Stacheldraht an der Engstelle ODER spare ich für das schwere MG der Home-Line?*
- **Referenz: GTFO** – Ressourcen sind extrem knapp und müssen im Team verhandelt werden ("Wer kriegt das Werkzeug-Auffüllpack?").

5. Technik: Babylon.js Performance & Netcode

Horde-Rendering (Nacht-Modus)

Browser-Animationen fressen CPU. 50+ individueller glTF-Skelett-Animationen bringen den Canvas zum Ruckeln.

- **Lösung:** Verwende für Horden **Thin Instances** oder **Instanced Meshes** in Kombination mit **Vertex-Animation-Textures (VAT)** (Animationen werden in Texturen gespeichert und auf der GPU berechnet). Damit renderst du 500+ Gegner flüssig bei 60 FPS.

Netcode-Architektur

Da deine Sim bereits entkoppelt ist (`src/sim/**`), ist die Basis perfekt.

- Da du kein Lockstep nutzt, fahre eine **Client-Prediction + Server-Reconciliation** Architektur.
- Bei 50 Entities: Repliziere **nicht** jeden Tick den vollständigen Zustand aller Einheiten. Repliziere Ziel-Punkte/Pfade der KI und schicke Korrekturen nur bei Abweichungen (Snapshot Interpolation).
- Nutze **Geckos.io** (UDP über WebRTC im Browser) statt WebSockets für die Netcode-Übertragung. TCP/WebSockets erzeugen Head-of-Line Blocking, was Multiplayer-Shooter unspielbar laggy macht.

Größter Risiko-Klumpen

Das "Niemandland vs. Graben"-Sichtlinien-Problem in First-Person.

Im Graben siehst du genau 5 Meter weit bis zur nächsten Kurve. Das erzeugt zwar tollen Grabenhorror, führt aber dazu, dass der Spieler keine Übersicht über die Schlacht hat. Schaut der Spieler über den Grabenrand (Parapet), sieht er das Niemandland. Wenn der Spieler dort auf Boxen/Kapseln schießt, fühlt sich das Spiel sofort flach an.

Wenn das Visuelle (Nebel, Mündungsfeuer, Silhouetten im Dunkeln/Rauch) nicht von Anfang an mitgedacht wird, scheitert das First-Person-Gefühl im Grabenkrieg komplett.

Die 3 wichtigsten nächsten Schritte

1. **Prototyper ein modulares Grid-Layout (Sektor):** Baue ein Tile-System (Straight, Corner, T-Junction, Parapet) und erstelle daraus ein simples Waypoint-Netzwerk für die Sim. Das löst Level-Gen und KI-Pathing gleichzeitig.
2. **Implementiere Vertex Animation Textures (VAT) / Instancing:** Teste frühzeitig, wie viele animierte Platzhalter-Gegner deine Pipeline auf einem Mittelklasse-Laptops schafft, bevor du das Gegner-Design finalisierst.
3. **Rückzugs-Mechanismus vertesten (Graybox):** Baue das Zonen-System (Front -> Verbindungsgraben -> Home-Line) mit einem temporären Artillerie-Sperrfeuer-Trigger, um zu prüfen, ob sich der "fechtende Rückzug" dynamisch und spannend anfühlt.

Perplexity

1. Der Grabensektor: Map-Design für First-Person

Einordnung: In First-Person ist Orientierung im Graben das Hauptproblem. Zu viele Verzweigungen, zu wenig Landmarks = Spieler verirrt sich oder fühlt sich verloren. Gleichzeitig brauchst du Tiefe (Front → Verbindung → Home) und skalierbare Breite für 1–4 Spieler.

Optionen:

Ansatz	Wie	Trade-offs
Modulare Hand-Bausteine (vorgefertigte Grabensegmente, T-Kreuzungen, Ecken, Bunker)	Baue einen Generator, der aus einem Graphen (Frontlinie als Hauptachse, Verbindungsgräben als Äste, Home-Line als Rückgrat) Module auswählt und zusammensetzt.	+ Kontrolle über Lesbarkeit, Engstellen, Deckung. – Mehr Art-Arbeit upfront, aber wiederverwendbar.
Vollprozedural mit Regeln (Noise + Constraints)	Generiere Grabenverläufe algorithmisch (z.B. Perlin-Noise für wellige Frontlinie), schneide Verbindungsgräben im festen	+ Hohe Varianz. – Risiko von unspielbaren Layouts, mehr Debug-Aufwand.

Ansatz	Wie	Trade-offs
	Rhythmus (alle 20–30 m), platziere Home-Line parallel im Abstand.	
Hybrid (handgebaute „Sektor-Templates“ + prozedurale Variation)	5–10 vorgebaute Sektor-Archetypen (breit/schmal, viele/wenige Verbindungsgräben), prozedural nur Details (Drahtpositionen, Sandsäcke, Trümmer).	+ Beste Kontrolle bei guter Varianz. – Erfordert mehr Designarbeit zu Beginn.

Empfehlung: Hybrid-Ansatz mit modularen Bausteinen. Warum? First-Person braucht konsistente Lesbarkeit: Schilder („A-7“, „B-3“), Kompassrose im HUD, und vor allem **wiedererkennbare Landmarks** (ein umgekippter Panzer, ein ausgebrannter Bunker, ein besonderer Grabenknicke). Module erlauben dir, diese Landmarks gezielt zu platzieren. Referenz: *Arma Reforger* Trench-Systeme zeigen, dass selbst in 3rd-Person modulare Systeme besser lesbar sind als rein prozedurale.reforger.armaplatform+1

Skalierung mit Spielerzahl: Breite der Frontlinie dynamisch anpassen (mehr Spieler = breitere Front, mehr Verbindungsgräben). Pro Spieler ~15–20 m Frontbreite als Daumenregel. Home-Line bleibt fix breit genug für alle.

2. Gegner-Vielfalt & KI: Wie viel reicht?

Einordnung: Du hast aktuell nur einen langsamen Nahkämpfer. Das ist ein guter Start, aber für Wave-Survival brauchst du **Druckvariation** (nah/fern, schnell/langsam, unterdrückend/stürmend).

Optionen:

KI-Tiefe	Was	Trade-offs
Minimal (geradliniger Anmarsch, kein Pathing)	Gegner laufen in direkter Linie zum nächsten Spieler, ignorieren Deckung.	+ Einfach. – Wirkt dumm, besonders bei MG/Grabengängern.
Einfaches Pathing (NavMesh oder Wegpunkte entlang der Gräben)	Gegner folgen Grabenverläufen, weichen Hindernissen aus, nutzen Deckung opportunistisch.	+ Glaubwürdiger. – Mehr Implementierungsaufwand.
Verhaltensbäume (Zustände: Anmarsch → Deckung → Sturm → Nahkampf)	Gegner reagieren auf Spieleraktionen (unterdrückt → suchen Deckung, flankieren).	+ Tiefes Gameplay. – Komplexität explodiert schnell.

Empfehlung: Einfaches Pathing mit Wegpunkten entlang der Grabenachsen. Warum? Gräben sind stark strukturiert — du kannst ein **Graph-basiertes Pathing** bauen: Jeder Grabenabschnitt ist ein Node, Verbindungen sind Kanten. Gegner folgen diesem Graphen, was deutlich glaubwürdiger wirkt als geradliniges Laufen, aber ohne volle NavMesh-Komplexität. Referenz: *Deep Rock Galactic* nutzt einfaches Graph-Pathing für Gegner in Höhlen — funktioniert auch in engen Räumen.[mightyprofessionalgaming](https://mightyprofessionalgaming.com/)

Priorisierte Gegner-Reihenfolge:

1. **Bajonett-Charger** (schnell, direkt) — erhöht Druck.
2. **MG-Trupp** (unterdrückt, zwingt Spieler in Deckung) — erzwingt Teamplay.
3. **Grabenräumer** (öffnet Parapet) — bedroht deine Platzierungen.
4. **Sturmtrupp** (jagt Platzierungen) — Konter gegen Bauen.

Pathing ist **notwendig** für alle außer dem Charger. Ohne Pathing wirken MG-Trupps und Grabengänger albern.

3. Rückzugs-Loop: Fechtender Rückzug lesbar machen

Einordnung: Das ist ein **kritisches Design-Risiko**. Wenn Spieler nicht verstehen, warum sie zurückweichen müssen oder wann die Home-Line fällt, fühlt es sich frustrierend an, nicht spannend.

Optionen:

Mechanik	Wie	Trade-offs
Visuelle Marker (Frontlinie als rote Zone auf Minikarte, Home-Line als grüne)	Spieler sehen live, welche Abschnitte gefallen sind.	+ Klar. – Erfordert UI-Arbeit.
Akustische Signale (Funkdurchsagen: „Abschnitt B-4 gefallen!“, „Rückzug auf Home-Line!“)	Audio-Feedback für Fortschritt.	+ Immersiv. – Kann untergehen.
Progressive Zerstörung (Grabenwände brechen, Draht wird gerissen, Bunker brennen)	Visuelles Feedback für Verlust.	+ Sehr lesbar. – Mehr Art/Shader-Arbeit.

Empfehlung: Kombination aus visuellen Markern + akustischen Signalen. Minikarte mit Farbcodierung (rot = gefallen, gelb = unter Angriff, grün = sicher) ist unverzichtbar. Dazu Funkdurchsagen („Kompanie, Abschnitt C-2 ist überrannt — Rückzug auf Home-Line!“). Referenz: *Helldivers 2* nutzt beides hervorragend — Spieler wissen immer, wo die Front ist. [flukz](#)

Spannung statt Frust: Mach den Rückzug **aktiv**, nicht passiv. Spieler müssen **Engstellen halten** (Verbindungsgräben), um Zeit zu gewinnen. Jeder gehaltene Verbindungsgraben verzögert den Home-Line-Verlust um X Sekunden. Das gibt Spielern Kontrolle.

4. Aktive Fähigkeit per In-Mission-Quest freischalten

Einordnung: Du willst, dass Spieler zur Home-Line laufen, um beim Feldkommandeur ihre Fähigkeit freizuschalten. Das ist ein **starker Immersions-Hebel**, aber auch ein **Flow-Killer**, wenn es zu oft passiert.

Optionen:

Methode	Wie	Trade-offs
In-Mission-Quest (zur Home-Line laufen, melden)	Spieler müssen physisch zurücklaufen.	+ Immersiv, erzwingt Rückzug. – Unterbricht Combat-Flow.
Automatisch nach Zeit/Wellen	Fähigkeit schaltet sich nach Welle X oder nach Y Minuten frei.	+ Kein Flow-Bruch. – Weniger immersiv.
Ressourcen-basiert (Nachschub ausgeben für Fähigkeit)	Spieler kaufen Fähigkeit im Einsatz.	+ Strategische Entscheidung. – Erfordert Ökonomie-Balance.

Empfehlung: Hybrid: Fähigkeit schaltet sich **automatisch nach Welle 3** frei, ABER Spieler müssen **einmal zur Home-Line laufen**, um sie zu **aktivieren**. Warum? Das gibt den immersiven „Meldung beim Kommandeur“-Moment, aber nur einmal pro Einsatz — nicht jede Welle. Referenz: *Deep Rock Galactic* nutzt ähnliche „Call-In“-Mechaniken (zur MULE laden, aber nur einmal pro Mission).

Alternative, die ich bevorzuge: Fähigkeit ist von Anfang an verfügbar, aber hat **3 Ladungen**, die nur an der Home-Line nachgeladen werden können. Das erzwingt Rückzug ohne Flow-Bruch.

5. Nachschub-Ökonomie: Eine oder zwei Währungen?

Einordnung: Zu viele Währungen = Mikromanagement-Hölle. Zu wenige = alle Entscheidungen fühlen sich gleich an.

Optionen:

Modell	Wie	Trade-offs
Eine Währung (Nachschub für alles)	Spieler sammeln eine Ressource, geben sie für Fähigkeiten, Platzierungen, Wandwaffen aus.	+ Einfach. – Entscheidungen können flach werden.
Zwei Währungen (Nachschub + Spezialressource)	Nachschub für Basis-Sachen, Spezialressource (z.B. „Pionier-Material“) für Bauen/Spreng.	+ Tiefere Entscheidungen. – Mehr Balance-Aufwand.
Klassen-spezifische Ressourcen	Jeder Klasse hat eigene Ressource (Sanitäter: Verbandsmaterial, Pionier: Sprengstoff).	+ Fördert Rollenidentität. – Kann Solo-Spieler bestrafen.

Empfehlung: Eine Währung, aber mit „Budgets“ pro Kategorie. Beispiel: Pro Einsatz hast du 100 Nachschub. Davon max. 40 für Platzierungen, 30 für Fähigkeiten, 30 für Wandwaffen. Das verhindert, dass Spieler alles in eine Kategorie stecken, aber hält die Ökonomie simpel. Referenz: *Rainbow Six: Siege* nutzt ähnliches „Gadget-Budget“-System.

6. Onboarding: Die erste Stunde

Einordnung: Die erste Stunde muss **Tutorial + Core-Loop** verbinden, ohne sich wie ein Tutorial anzufühlen.

Empfohlener Ablauf:

1. **Einsatz 1 (Tag, Welle 1–3):** Nur Schütze-Klasse, nur Primärwaffe. Gegner: nur Linieninfanterie. Ziel: Überlebe 3 Wellen, extrahiere. (10 Min)
2. **Quartier:** Erster Soldat bekommt XP, schalte MG-Schütze frei. (2 Min)
3. **Einsatz 2 (Tag, Welle 1–5):** Wähle Schütze oder MG-Schütze. Gegner: +MG-Trupp. Ziel: Halte Frontlinie, MG aufstellen lernen. (15 Min)
4. **Quartier:** Schalte Pionier frei, erste Platzierung (Sandsäcke). (2 Min)
5. **Einsatz 3 (Nacht, Welle 1–4):** Erste Nacht-Erfahrung, nur Wiedergänger. Ziel: Überlebe mit Taschenlampe, lerne Angst-Mechanik. (15 Min)
6. **Quartier:** Schalte erste aktive Fähigkeit frei (z.B. Pionier: Grabenreparatur). (2 Min)
7. **Einsatz 4 (Tag, Welle 1–6):** Alle 3 Klassen verfügbar, erste Platzierungen nutzbar. (20 Min)

Gesamt: ~60–70 Min, Spieler haben alle Kernmechaniken gesehen, ohne überfordert zu sein.

7. Koop-Design: 4 Rollen, echtes Zusammenspiel

Einordnung: Das größte Risiko: Koop fühlt sich an wie „4× Solo“, oder Solo wird zum „4-Spieler-Balance-Albtraum“.

Optionen:

Ansatz	Wie	Trade-offs
Harte Rollen-Abhängigkeit (Sanitäter muss beleben, Pionier muss reparieren)	Ohne richtige Rolle scheitert das Team.	+ Echtes Teamplay. – Kann Solo unmöglich machen.
Weiche Synergien (jede Rolle kann alles, aber mit Bonus)	Schütze kann beleben (langsamer), Pionier kann schießen (weniger genau).	+ Solo bleibt stark. – Rollenidentität leidet.
Hybrid (Basis-Aktionen für alle, Signatur für Rolle)	Jeder kann verbinden, aber nur Sanitäter hat schnelle Belegung.	+ Beste Balance. – Erfordert feine Balance.

Empfehlung: Hybrid. Jeder kann jede Basis-Aktion (beleben, reparieren, bauen), aber **nur die Signatur-Klasse hat die volle Effizienz**. Beispiel: Schütze kann beleben (10 Sek), Sanitäter (3 Sek + HP-Bonus). Pionier kann reparieren (langsam), aber nur Pionier kann Draht/Sandsäcke platzieren. Referenz: *Deep Rock Galactic* macht das perfekt — jeder kann alles, aber die Spezialisten sind deutlich besser. [flukz](#)

Solo-Balance: Solo-Spieler bekommt **KI-Trupps** (2–3 Bot-Soldaten), die Basis-Aktionen ausführen, aber keine Signature. Das hält Solo spielbar, ohne Koop zu entwerten.

8. Prozedurale Grabengenerierung (Technik)

Empfehlung: Graph-basierte Generierung mit modularen Segmenten.

Ablauf:

1. **Frontlinie generieren:** Perlin-Noise oder Random-Walk für wellige Linie (Länge = Spielerzahl $\times$ 20 m).
2. **Home-Line parallel:** Im Abstand von 50–80 m, gleiche Länge.
3. **Verbindungsgräben:** Alle 20–30 m senkrecht zwischen Front und Home, mit leichten Kurven.
4. **Module auswählen:** Für jeden Grabenabschnitt zufällig ein Modul (gerade, Kurve, T-Kreuzung, Bunker) aus Pool.
5. **Landmarks platzieren:** 3–5 pro Sektor (Panzerwrack, Bunker, Artilleriekrater).

Kollision/Navigation: Generiere gleichzeitig ein **NavGraph** (Nodes = Grabenabschnitte, Kanten = Verbindungen). Das dient sowohl für KI-Pathing als auch für Minikarte.

Referenz: *Sammys Trench System* (Arma Reforger) nutzt genau diesen Ansatz — Spline-basierte Generierung mit modularen Segmenten. [reforger.armaplatform](https://reforger.armaplatform.com/)

9. Koop-Netcode: Server-autoritativ + State-Replikation

Einordnung: Du hast bereits die richtige Entscheidung getroffen: **kein Lockstep, sondern State-Replikation**. Für einen Wave-Shooter mit ~50 Entities ist das der Standard.

Empfohlene Architektur:

Komponente	Empfehlung
Tick-Rate	30 Hz Server (60 Hz wenn Bandbreite es zulässt)
Client-Prediction	Nur für Spieler-Input (Bewegung, Schießen) — server korrigiert bei Diskrepanz
Entity-Interpolation	100–150 ms Buffer für andere Spieler/Gegner
Snapshot-Delta	Server sendet nur geänderte Entities pro Tick
Input-Kompression	4 Bytes pro Player-Input (Position + Rotation + Action-Bits)

Node-Server-Setup:

- **WebSocket** für Echtzeit-Kommunikation (Socket.io oder ws)
- **Autoritativer Sim-Loop** (gleicher Code wie Client-Sim, aber ohne Rendering)
- **State-Replikation:** Serialisiere Sim-State (JSON oder Binary), sende an Clients
- **Client:** Empfängt State, interpoliert zwischen Snapshots

Referenz: *Burncode* und *Meseta* erklären detailliert, warum State-Replikation für FPS/Co-op-Shooter besser ist als Lockstep. [burncode+2](#)

Risiko: Bandbreite bei 4 Spielern + 50 Gegnern. Teste früh mit **Entity-Culling** (nur relevante Entities pro Client senden).

10. Rendering-Skalierung in Babylon.js

Empfehlung: Thin Instances für Gegner, LOD für Details, Frustum-Culling.

Technik	Wann
Thin Instances	100+ identische Gegner (Nacht-Horde) — packt Matrizen in Buffer, minimaler CPU-Overhead forum.babylonjs
Regular Instances	10–100 Gegner mit Variation (verschiedene Texturen/Animationen)
LOD	Gegner in der Ferne = niedrigeres Poly-Modell
Frustum-Culling	Babylon.js macht das automatisch, aber achte auf korrekte Bounding-Boxes

Grenzen im Browser: ~500–1000 Thin-Instance-Gegner bei akzeptabler Framerate (60 FPS auf Mittelklasse-PC). Teste früh auf schwächerer Hardware (Laptop iGPU).

Referenz: Babylon.js Performance-Guide empfiehlt Thin Instances für Massen-Rendering.[forum.babylonjs](https://forum.babylonjs.com/)+1

11. Bundle-Größe: Babylon.js optimieren

Einordnung: 1,5 MB gzip für @babylonjs/core ist akzeptabel, aber es geht kleiner.

Optionen:

Methode	Einsparung Aufwand	
Tree-Shaking (nur genutzte Module importieren)	30–50%	Mittel
Code-Splitting (Lobby/Quartier separat laden)	20–30%	Hoch
Babylon.js Serialisierung (nur Core, kein GUI, kein Loader)	40–60%	Mittel

Empfehlung: Tree-Shaking + minimale Core-Imports. Importiere nur @babylonjs/core Module, die du nutzt (Mesh, Material, Camera, Light). Vermeide @babylonjs/loaders, @babylonjs/gui im Haupt-Bundle. Code-Splitting lohnt erst, wenn Lobby/Quartier >500 KB haben.

12. Art-Pipeline: Schematischer Stil

Empfehlung: Flat-Shaded Low-Poly + gedämpfte Palette.

Asset-Typ

Empfehlung

Modelle glTF 2.0, 500–2000 Tris pro Charakter, 100–500 Tris pro Gegner

Texturen 512×512 Atlas pro Klasse, KTX2-Kompression

Shader Flat-Shading (kein Lighting), Farbvariation per Vertex-Color

Wie viel Shader statt Assets? Viel. Du kannst **Farbvariation, Abnutzung, Blut (schematisch)** per Shader machen statt neuer Texturen. Referenz: *Battlefield 1* (schematischer Look in bestimmten Modi) nutzt ähnliche Techniken.

13–15. Backlog, Priorisierung, Risiken

Überflüssiges (vorerst):

- **Sturmtruppler-Klasse** (Backlog) — erst die 4 Grundklassen polieren.
- **Wandwaffen-Vielfalt** — 2–3 reichen für v1.
- **Nationen-Traits** — kosmetisch, kein Gameplay-Hebel.

Fehlt:

- **Sound-Design** (kritisch für Horror/Nacht).
- **UI/UX-Tests** (Onboarding-Flow).
- **Performance-Tests** (Browser auf schwacher Hardware).

Empfohlene Reihenfolge:

1. **Modulare Graben-Bausteine + Graph-Pathing** (AP3)
2. **Gegner-Vielfalt (4 Typen) + einfaches KI-Pathing** (AP4)
3. **Rückzugs-Loop + Home-Line-Mechanik** (AP5)
4. **Klassen + Signature-Fähigkeiten** (AP6)
5. **Quartier + Nachschub-Ökonomie** (AP7)
6. **Koop-Netcode (2 Spieler)** (AP8)

Größter Risiko-Klumpen: Koop-Netcode + State-Replikation. Wenn das nicht performant läuft (Bandbreite, Latenz), kippt das Fernziel. Teste das **früh** (nach AP5), nicht am Ende.

Meine 3 wichtigsten nächsten Schritte

1. **Modulare Graben-Bausteine + Graph-Pathing implementieren** (AP3) — das ist die Basis für alles Weitere (Map, KI, Rückzug).
 2. **4 Gegner-Typen + einfaches Pathing** (AP4) — ohne Gegner-Vielfalt kein Wave-Survival.
 3. **Koop-Netcode-Prototyp (2 Spieler)** (AP8) — früh testen, ob State-Replikation im Browser performant läuft.
-

Wo ich widerspreche:

- **In-Mission-Quest für Fähigkeit:** Zu flow-brechend. Hybrid-Lösung (automatisch freischalten, einmal zur Home-Line zum Aktivieren) ist besser.
- **Vollprozedurale Map:** Zu riskant für First-Person-Orientierung. Hybrid (Templates + Variation) ist sicherer.
- **Eine Währung für alles:** Budget-Kategorien sind nötig, sonst werden Entscheidungen flach.

ChatGPT:

Mein Ausgangspunkt: Die Grundidee trägt. Aber dein größtes Risiko ist nicht Babylon, Netcode oder Bundle-Größe. Es ist, dass du im Moment noch **nicht den eigentlichen Kern von „Halt die Linie“ testest**. Der aktuelle Build testet FPS-Bewegung, Hitscan und einen Wave-Director gegen langsame Nahkämpfer; noch fehlen echter Sektor, Rückzug, Fernkampfgegner, Tag/Nacht und taktische Ausgaben.

Dadurch besteht die Gefahr, technisch sauber sehr weit zu bauen, bevor klar ist, ob das zentrale Versprechen — **Front halten, Boden verlieren, geordnet zurückfallen, an der Home-Line überleben** — tatsächlich Spaß macht. Genau dieses Versprechen würde ich jetzt isolieren.

1. Sektor: nicht vollprozedural bauen

Der dreistufige Aufbau Frontlinie → Verbindungsgraben → Home-Line ist aus meiner Sicht richtig, aber in First Person gefährlich: Ein realistisches Grabennetz wird schnell entweder ein **Schlauch** oder ein **Labyrinth**. Dein Design verlangt zusätzlich, dass einzelne Frontabschnitte fallen und rückerobert werden können.

Option A — komplett handgebaut

Plus: maximale Kontrolle über Sichtlinien, Landmarken, Rückzugswege und Encounter-Pacing.
Minus: wenig Wiederspielwert, hoher Content-Aufwand.

Option B — vollständig prozedural

Plus: theoretisch hohe Variation.
Minus: enormes Balancingproblem. Gerade Sichtlinien, Spawnpunkte, MG-Felder, Engstellen und Rückzugswege müssen gleichzeitig funktionieren. Ein formal gültiger Sektor ist noch lange kein guter FPS-Level.

Option C — authored modules + prozedurales Makrolayout

Das würde ich nehmen.

Baue beispielsweise:

- 6–10 Frontmodule,
- 4–6 Verbindungsgrabenmodule,
- 3–4 Home-Line-Varianten,
- definierte Kreuzungen, Bunker, Traversen und Ausweichstellen.

Jedes Modul bekommt semantische Metadaten:

`connections, coverSlots, enemyEntrances, defenseAnchors, landmarks, navLanes, buildSlots.`

Der Generator entscheidet dann über die **Anordnung**, nicht über jedes einzelne Stück Geometrie.

Das ist ungefähr die Lektion, die ich eher aus **Deep Rock Galactic** als aus klassischen Roguelikes ziehen würde: Prozedurale Räume dürfen variieren, aber Spieler brauchen markante Strukturen und verständliche Makrogeometrie.

Navigation

Eine Minimap würde ich zunächst **nicht** als Reparatur für verwirrendes Leveldesign einsetzen.

Stärker wären:

- deutlich unterschiedliche Silhouetten der drei Tiefenzonen;
- nummerierte Frontsektoren: A, B, C;
- farbige/typografische Feldschilder;
- Kommunikationsdraht, Lampen oder Pfosten als „Spine“ zur Home-Line;
- extrem erkennbare Home-Line-Silhouette;
- Kompass mit „HOME“ und aktiven Sektor-Markern.

Wenn du nach zehn Minuten eine Minimap brauchst, damit Tester zurück zur Home-Line finden, ist wahrscheinlich der Sektor zu kompliziert.

Spielerzahl würde ich nicht über Mapbreite skalieren. Halte die Geometrie weitgehend stabil und skaliere stattdessen aktive Angriffsachsen, Spawnmenge und gleichzeitig bedrohte Frontabschnitte. Sonst musst du faktisch mehrere Levelsets balancieren.

2. Gegner-KI: weniger Intelligenz, mehr unterschiedliche Verben

Hier würde ich deine Planung deutlich kürzen.

Für das Spiel sind nicht zehn intelligente Gegnertypen wichtig. Wichtig ist, dass Gegner den Spieler zu **unterschiedlichen Entscheidungen** zwingen. Du planst bereits Linieninfanterie, Sturmtrupp, MG, Grabenräumer und Panzer sowie ein separates Nacht-Roster.

Ich würde für den nächsten Prototyp nur drei Verhaltensweisen bauen

Linieninfanterist:

rückt von Deckung zu Deckung vor, bleibt gelegentlich stehen, schießt.

Stürmer:

ignoriert teilweise Deckung und versucht, in den Graben zu kommen.

Disruptor:

greift nicht primär HP an, sondern deine Verteidigungsstruktur — etwa ein Grabenräumer oder Platzierungsjäger.

Damit hast du schon:

- Positionsdruck,
- Zeitdruck,
- Zielpriorisierung.

Mehr braucht der Kern zunächst nicht.

Pathfinding

Ich würde noch **kein allgemeines Navmesh-System** bauen.

Dein Grabensystem ist ein Sonderfall mit klar definierten Korridoren. Nutze zunächst einen semantischen Graphen:

spawn → approach lane → breach → trench segment → connector → home

Innerhalb eines Moduls kann ein Gegner ein paar lokale Wegpunkte haben.

Das hat einen wichtigen Nebeneffekt: Derselbe Datensatz kann später Director, Spawnlogik und serverseitige KI antreiben. Das passt sehr gut zu deiner ohnehin strikten Trennung zwischen headless Sim und Renderer.

Ein vollwertiges dynamisches Pathfinding würde ich erst einführen, sobald dir ein konkreter Gegnermechanismus beweist, dass du es brauchst.

3. Der Rückzug ist dein eigentliches Alleinstellungsmerkmal

Hier würde ich den größten Designaufwand investieren.

Der Rückzug darf sich **nicht wie verlorener Fortschritt** anfühlen.

Schlechte Variante

Front besitzt 100 HP → fällt → Spieler verliert Gebiet → alles wird schwieriger.

Dann empfindet der Spieler jeden Rückzug einfach als Failure-State in Zeitlupe.

Bessere Variante

Jede Verteidigungstiefe verändert das Gefecht.

Beispielsweise:

Frontlinie

- beste Sichtlinien;
- maximaler Ressourcen-Ertrag;
- weit auseinanderliegende Angriffssachsen.

Verbindungsgraben

- extreme Engstellen;
- Nahkampf gefährlicher;
- MG besonders stark;
- weniger Ressourcenquellen.

Home-Line

- starke vorbereitete Verteidigung;
- Munitionslager;
- Verbandsplatz;
- aber mehrere Zugänge gleichzeitig und finale Uhr.

Damit bedeutet Rückzug:

„Wir verlieren strategisch, gewinnen aber taktisch eine neue Situation.“

Das ist entscheidend.

Noch wichtiger: Verlust muss Kosten verursachen

Frontabschnitte könnten enthalten:

- Nachschubdepots,
- Beobachtungsposten,
- Munitionskisten,
- Artilleriebeobachter,
- Extraktionsboni.

Dann tut das Aufgeben eines Abschnitts weh, ohne dass es automatisch eine Todesspirale erzeugt.

Rückeroberung sollte selten sein. Wenn Spieler ständig hin- und hercapturen, wird aus „fechtend zurückziehen“ schnell Battlefield-Domination im Graben.

Mein Modell wäre eher:

stabil → bedrängt → gebrochen → verloren

Nach „verloren“ ist Rückeroberung ein teures optionales Gegenmanöver.

4. Ein Designwiderspruch: harte Elite-Konter vs. dauerhaft spielbares Solo

Das würde ich jetzt lösen.

Du sagst einerseits, dass Solo dauerhaft voll spielbar bleiben soll; andererseits sollen Panzer und Koloss harte Konter besitzen und Spreng-/AT-Ausrüstung verlangen.

Mit minimalem Primär-/Sekundär-Loadout kann daraus entstehen:

Spieler trifft Elite → besitzt falsche Klasse/Ausrüstung → Encounter faktisch nicht lösbar.

Das wäre kein taktischer Konter, sondern eine Loadout-Falle.

Ich würde deshalb unterscheiden:

Soft requirement: Spezialist löst Problem effizient.

Hard requirement: Spezialressource muss im Level garantiert verfügbar sein.

Beispiel Panzer:

- Pionier besitzt bereits Sprengladung → bester Konter.
- Andere Klassen können eine seltene AT-Wandwaffe oder Feldladung holen.
- Alternativ existiert ein Umgebungskonter wie Artilleriesignal oder Sprengstoffdepot.

So bleibt der Pionier wertvoll, ohne dass die Lobbyzusammenstellung über Sieg/Niederlage entscheidet.

Das ist auch meine Grundregel für Koop:

Rollen sollen Krisen **besser lösen**, nicht anderen Spielern die Erlaubnis geben, sie überhaupt lösen zu dürfen.

5. Klassen: deine vier Rollen sind gut, aber aktuell zu funktionsgebunden

Schütze, MG, Pionier und Sanitäter haben verständliche Fantasien.

Ich würde sie allerdings nicht wie MMO-Klassen bauen.

Die beste Struktur wäre meiner Meinung nach:

Jeder kann: schießen, improvisiert heilen, Munition aufnehmen, grundlegende Interaktionen ausführen.

Spezialist kann es stark:

Sanitäter revivet erheblich schneller und betreibt Verbandsplatz.

Pionier repariert und baut richtige Befestigungen.

MG-Schütze kontrolliert eine Angriffslinie.

Schütze bekommt Mobilität/Präzision/Aufklärung.

So funktioniert Solo automatisch besser.

Aktive Fähigkeit über In-Mission-Quest

Als dauerhafte Mechanik halte ich das für eine **schlechte Idee**.

Beim ersten Mal ist:

„Melde dich beim Feldkommandeur“

schönes Onboarding und Worldbuilding.

Beim 47. Einsatz ist es vermutlich nur noch eine Pflicht-Rennstrecke.

Ich würde daraus entweder

1. einen einmaligen Klassen-Unlock machen oder
2. einen optionalen Feldauftrag machen, der die Fähigkeit **verbessert**, statt sie überhaupt erst verfügbar zu machen.

Die Basisklasse sollte nach dem Loadout unmittelbar vollständig funktionieren.

6. Nachschub: eine Währung, aber nicht ein einziger Entscheidungsraum

Zwei taktische Währungen würde ich vermeiden.

Wenn Spieler gleichzeitig „Baumaterial“, „Munitionspunkte“, „Kommandopunkte“ usw. verwalten, wird die Gefechtssituation schnell zu Buchhaltung.

Ich würde zunächst **einen Nachschubpool** verwenden.

Aber nicht alles darf beliebig gegeneinander austauschbar sein.

Beispiel:

- Sandsack: 15 Nachschub
- Munitionskiste: 30
- MG-Stellung: 50
- KI-Trupp: 60

Zusätzlich limitierst du über **Slots und Verfügbarkeit**:

- nur zwei schwere Platzierungen pro Frontabschnitt;
- ein KI-Trupp gleichzeitig;
- Wandwaffe nur an bestimmten Depots.

Dann entsteht interessante Knappheit durch zwei Achsen:

Kann ich es bezahlen?

und

wofür verbrauche ich meinen knappen Slot?

Das ist stärker als eine zweite Währung.

Meta-Fortschritt im Quartier darf dagegen problemlos eine andere Ressource besitzen, weil das ein eigener Entscheidungskontext ist.

7. Die erste Stunde würde ich wesentlich kleiner machen als deine Feature-Liste

Das aktuelle Konzept enthält enorm viele Systeme.

Meine erste Stunde:

0–10 Minuten

Schütze. Gewehr. Ein Frontabschnitt. Schießen, Reload, Munition, Deckung.

10–25 Minuten

Drei Frontabschnitte. Gegner greifen an zwei Stellen an. Spieler lernt anhand von Signalen: „Diese Linie bricht.“

25–40 Minuten

Erster garantierter Rückzug. Der Spieler erlebt den Verbindungsgraben und verteidigt die Home-Line.

40–50 Minuten

Erste taktische Nachschubentscheidung: Munition **oder** Feldbefestigung.

50–60 Minuten

Erste Klassenalternative oder kurzer Nacht-Einsatz.

Noch kein Quartierbaum mit zwölf Upgrades.

Keine fünf Klassen.

Keine zehn Gegner.

Keine komplexe Nationenprogression.

Nach dieser Stunde sollte der Spieler erklären können:

„Ich hielt eine Front, verlor sie, zog mich zurück und rettete den Sektor.“

Wenn genau dieser Satz Spaß gemacht hat, lohnt sich der Rest.

8. Tag und Nacht sind wahrscheinlich dein größter Scope-Multiplikator

In deinem Pitch ist die Trennung hervorragend: tagsüber Armee mit Rückfeuer und Deckung, nachts Horde und Horror.

Produktionstechnisch baust du damit aber fast **zwei Kampfsysteme**:

Tag benötigt:

- ranged AI,
- Cover,
- Suppression,
- Sichtlinien,
- taktisches Vorrücken.

Nacht benötigt:

- Massenspawns,
- Nahkampfnavigation,
- Crowd Management,
- besondere Lesbarkeit bei Dunkelheit,
- anderes Audio/Pacing.

Das würde ich nicht streichen — es ist vermutlich Teil des Hooks.

Aber ich würde **nicht gleichzeitig beide Roster entwickeln**.

Baue:

- einen überzeugenden Tag-Gegner;
- einen überzeugenden Nacht-Gegner.

Dann spiele denselben Sektor zweimal.

Wenn sich diese beiden Runs tatsächlich wie zwei Seiten desselben Spiels anfühlen, hast du einen sehr wichtigen Risikopunkt geklärt.

9. Technik: deine Architekturentscheidung ist vernünftig — jetzt darf sie aber kein Selbstzweck werden

Die saubere `src/sim/**`-Grenze mit Commands, Seed und `dt`, getrennt vom Renderer, ist genau die richtige Vorbereitung für einen späteren autoritativen Server.

Ich würde trotzdem **jetzt keinen Netcode bauen**.

Auch nicht „nur schon mal das Grundgerüst“.

Für ~50 spielrelevante Entities ist nicht die reine Entityzahl dein schwerstes Problem. Schwer sind später:

- Ownership;
- Prediction;
- Hit-Validation;
- Reconciliation;
- Entity Lifetime;
- Paketverlust/Latenz;
- Zustandskompression.

Aber all das wird leichter, wenn deine Sim bereits klar definierte Inputs und States besitzt.

Daher jetzt nur netcodefreundlich bleiben:

- stabile Entity IDs;
- serialisierbare Commands;
- keine Gameplay-Logik im Renderer;
- explizite Spawn/Despawn-Ereignisse;
- klarer authoritative state.

Mehr nicht.

Dass Koop Fernziel und noch ohne Netcode ist, ist derzeit kein Defizit.

10. Rendering, Art und Bundle-Größe würde ich fast komplett ignorieren

Du hast bereits einen Boxen-Testgraben und Platzhalterdarstellung.

Perfekt.

Ich würde vorerst weder:

- ernsthaft LOD-Systeme bauen,
- KTX2-Pipeline perfektionieren,
- Shader-Art entwickeln,
- noch viel Zeit investieren, Babylon um einige hundert Kilobyte kleiner zu bekommen.

Diese Arbeit ist nur wertvoll, nachdem dein tatsächlicher Sektor existiert.

Thin Instances, Atlases usw. lösen Performanceprobleme. Momentan weißt du noch gar nicht, **welches** Performanceproblem dein echtes Spiel haben wird.

Optimieren nach Messung.

Was ich aus dem Backlog streichen oder verschieben würde

Deine bewusst gesetzten Grundentscheidungen — FPS, 3D, kein Tower Defense, kein Permadeath, Kader-Modell — würde ich tatsächlich nicht neu aufrollen.

Für deutlich später würde ich dagegen schieben:

- Sturmtruppler-Klasse;
- vollständiges 5+5-Gegnerroster;
- Panzer;
- komplexes Quartier;
- Nations-Vertrautheit;
- seltene exotische Wandwaffen;
- Art-Pipeline-Optimierung;
- Multiplayer;
- „Krieg“-Modus sowieso;
- ausgefeilte prozedurale Variation.

Insbesondere **Nations-Vertrautheit** riecht für mich aktuell nach System, das Zahlen erzeugt, aber noch keine spannende Entscheidung erzeugt. Das wäre einer meiner ersten Streichkandidaten.

Mein größter Risiko-Klumpen

Nicht Netcode.

Nicht Browserperformance.

Nicht Contentmenge.

Der Rückzugs-Loop + Leveldesign.

Denn wenn der Spieler am Ende einfach nur einen engeren Korridor nach hinten läuft und anschließend dieselben Gegner an einer anderen Barrikade abschießt, ist das zentrale Feature funktional vorhanden und spielerisch bedeutungslos.

Du brauchst den Moment:

„Links ist eingebrochen. Wir können noch zwei Minuten um den Abschnitt kämpfen — oder Munition mitnehmen und die Kreuzung dahinter vorbereiten.“

Das ist „Halt die Linie“.

Wenn dieser Entscheidungsmoment funktioniert, tragen Klassen, Koop, Wirtschaft und Meta darauf auf.

Die drei nächsten Arbeitspakete, die ich bauen würde

1. AP3 — Verteidigung in der Tiefe

Eine einzige handgebaute Greybox:

Front A + Front B + Front C → zwei Connectoren → Home-Line

Dazu:

- Abschnittsbesitz;
- Breach-State;
- automatische Rückzugslogik;
- Verlust von Front-Boni;
- Home-Line-Finale.

Abnahmekriterium: Ein Tester versteht ohne Erklärung, wann eine Stellung verloren ist und wohin er zurückfallen soll.

2. AP4 — Zwei Kampfsprachen

Nur:

Tag: Rifleman mit Deckung + Schuss.

Nacht: aggressiver Wiedergänger.

Derselbe Sektor, derselbe Wave-Director, anderes Verhalten.

Abnahmekriterium: Beide Einsätze verlangen sichtbar unterschiedliche Positionierung und Prioritäten.

3. AP5 — erste echte Entscheidung

Füge exakt **eine Klasse** jenseits des Schützen und **drei Nachschubausgaben** hinzu.

Mein Kandidat wäre der Pionier:

- Sandsack,
- Draht,
- Munitionskiste oder Sprengladung.

Damit testest du gleichzeitig Klasse, Gelände, Wirtschaft und Solo-Tauglichkeit.

Abnahmekriterium: Spieler diskutieren nach dem Run darüber, *wofür* sie Nachschub hätten ausgeben sollen — nicht darüber, dass ihnen Nachschub gefehlt hat.

Wenn diese drei APs funktionieren, hast du aus meiner Sicht zum ersten Mal nicht nur einen Shooter-Prototyp, sondern einen **Prototyp von „Halt die Linie“ selbst**. Das entspricht auch genau der gewünschten Priorisierung auf konkret überprüfbare Mechaniken statt breiter Feature-Abdeckung.